Weekly Homework 04: Database Practice

Course: Python Programming
Textbook: *Murach's Python Programming*, 2nd edition
Required work: Chapter 17, Exercise 17-2, page 509
Submission: Canvas; due date is posted in Canvas
Python: 3.10 or newer; standard-library `sqlite3`
Total: 100 points

Purpose

Complete the assigned Chapter 17 textbook exercise to practice using Python with a database. This is individual textbook homework, separate from the Party Roster in-class lab. Do not add RPG features or other requirements that are not part of the assigned exercise.

Read first (part of the homework)

Before you start the exercise, read these pages of your textbook. Page numbers are the printed numbers at the bottom of each page. Reading is part of the assignment, and you should refer back to these pages while you work.

Pages	What to read	How it helps with the exercise
470-471	An introduction to relational databases	Tables, rows, and columns: what the Movie table holds.
476-481	The SQL statements	`SELECT` with `WHERE` and `ORDER BY` for the running-time query; `DELETE` for removing a movie.
486-491	Using Python with a database	Connecting, executing `SELECT` statements, and getting rows with `fetchone()` and `fetchall()`.
492-495	Executing `INSERT`, `UPDATE`, and `DELETE`, and testing database code	Why `commit()` is needed after a change, and how to check your database code.
496-505	The Movie List program	The program you are enhancing: how the database tier and the user interface fit together.

Optional warm-up: Exercise 17-1 on page 508 reviews a SQLite database.

In your journal, add a short **Reading notes** section before your step-by-step entries. In your own words, write two or three sentences about what you learned from the reading and one question or unclear point you had. Do not copy text from the book.

Read and complete

Complete **Exercise 17-2, steps 1-7**, using the directions in your textbook. The exercise has you review and test the Movie List program, improve the delete interaction, and add a command that finds movies below a requested running time.

Use one of these starting points:

- The course-provided files in `activity_4_party_roster/homework/chapter_17/` (open a terminal in that folder and run `python ui_starter.py`).
- A copy of the textbook's Chapter 17 Movie List exercise files, if you have downloaded them.

Choose one set of files and work on a copy. Do not combine the two starting points or use instructor solutions. The course starter is original course scaffolding for the textbook exercise; it does not replace the textbook directions.

What to submit

Submit through Canvas:

1. **Completed Python source files** for the assigned exercise. If you use the course starter, submit the completed `objects_starter.py`, `db_starter.py`, and `ui_starter.py`.
2. **One exercise journal**, organized by the assigned numbered steps. For each step, briefly record what you changed or tried, what happened when you ran or checked it, and how you responded to any problem. If it worked, describe what you observed.
3. **Run information in the journal:** identify the exercise and filenames, your Python version, any dependencies, and how to run the program.
4. **Testing evidence in the journal:** record the actions or inputs and the results you observed. Include an ordinary use case and relevant invalid or boundary cases.
5. **AI-use note in the journal:** name any tool you used, describe what you asked, identify a suggestion you accepted, changed, or rejected, and explain how you checked it. If you did not use AI, say so.

AI may be used to explain an error, review a small part of your own code, or suggest a test. Do not ask AI to complete the textbook exercise for you or submit code you cannot explain. Check suggestions by running your program and record meaningful assistance in the journal.

The journal may be Canvas text or an uploaded document when Canvas accepts that file type. If uploads are limited to `.py`, put the journal in the Canvas text entry or submission comments. Screenshots are optional; clear written observations are enough.

No README is required. Do not publish textbook homework to Git. Keep run information, testing notes, takeaways, and the AI-use note together in the journal. Do not submit the textbook PDF, instructor solutions, classmates' work, passwords, or other private information. The starter creates its own local sample database; do not submit that generated database file.

Checks to record

Run your completed program and record what you observed when you:

- View the current movie list.
- Use the delete command and decline confirmation; verify that the movie remains.
- Use the delete command and confirm; verify that the selected movie is removed.
- Request movies below a running-time limit; verify that the results are ordered by running time.
- Try an ID that is not in the database and a useful running-time boundary.

Use the textbook's instructions for the expected program behavior. Do not report a test as passed unless you actually tried it.

Rubric: 100 points

Criterion	Points	Full-credit evidence
Exercise 17-2: database lookups	10	The database layer returns the requested movie and the matching movie objects.
Exercise 17-2: confirmed deletion	15	The program asks before deletion and deletes only when the user confirms.
Exercise 17-2: running-time query	15	The database query selects movies below the requested limit and sorts them by running time.
Exercise 17-2: command and display	20	The menu and user interface support the new command and present its results clearly.
Exercise journal	25	Assigned-step coverage and actions (10); observations and learning (10); problem/fix or successful-check explanation and AI-use/non-use note (5).
Testing evidence	10	Ordinary and relevant invalid/boundary checks are recorded with actual results.
File organization and run information	5	Source files and any necessary sample data are identified (2); Python version and usable run/dependency information are in the journal (3).
Total	**100**	

Partial credit is available for documented attempts and demonstrated understanding. The journal, testing, and run details can all be in the same journal; no separate README or testing document is required. Follow the course-wide Homework Submission Expectations.